LAB REPORT: HEAP SORT PERFORMANCE ANALYSIS

Course: Design & Analysis of Algorithms

Name: Aryan Nautiyal **Roll Number:** 251210015

1. Aim

To understand, implement, and analyze the performance of **Heap Sort**. Sort a given set of elements using the Heap Sort algorithm (with and without the heapify method) and determine the runtime required across scaling array lengths N and data distributions using `<chrono>`.

2. Theory

Variant	Build Heap	Sort Phase	Worst Case	Space
Heap Sort (With Heapify)	$O(N)$	$O(N \log N)$	$O(N \log N)$	$O(\log N)$ aux
Heap Sort (Without Heapify)	$O(N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$ aux

- **With Heapify Method (Bottom-Up Build):** Constructs the initial max-heap by calling `heapify` starting from the last non-leaf node down to the root node (`n/2 - 1` down to `0`).
- **Without Heapify Method (Inline sift down):** Constructs the max-heap in-place by applying iterative sift-down logic directly within nested loops, starting from the last non-leaf parent node down to the root.

3. Source Code

```cpp
#include "test.hpp"
#include <algorithm>
#include <chrono>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>

void heapify(std::vector<int> &arr, size_t n, size_t i) {
  size_t largest = i;
  size_t left = 2 * i + 1;
  size_t right = 2 * i + 2;
  if (left < n && arr[left] > arr[largest])
    largest = left;
  if (right < n && arr[right] > arr[largest])
    largest = right;
  if (largest != i) {
    std::swap(arr[i], arr[largest]);
    heapify(arr, n, largest);
  }
}

void heapSortWithHeapify(std::vector<int> &arr) {
  size_t n = arr.size();
  if (n <= 1)
    return;
  for (int i = (n) / 2 - 1; i >= 0; --i) {
    heapify(arr, n, (i));
  }
  for (size_t i = n - 1; i > 0; --i) {
    std::swap(arr[0], arr[i]);
    heapify(arr, i, 0);
  }
}

void heapSortWithoutHeapify(std::vector<int> &arr) {
  int n = arr.size();
```

```
  for (int i = n / 2 - 1; i >= 0; i--) {
    int curr = i;
    while (true) {
      int largest = curr;
      int left = 2 * curr + 1, right = 2 * curr + 2;
      if (left < n && arr[left] > arr[largest]) {
        largest = left;
      }
      if (right < n && arr[right] > arr[largest]) {
        largest = right;
      }
      if (largest == curr) {
        break;
      }
      std::swap(arr[largest], arr[curr]);
      curr = largest;
    }
  }
  for (int i = n - 1; i > 0; i--) {
    std::swap(arr[0], arr[i]);
    int curr = 0;
    while (true) {
      int largest = curr;
      int left = 2 * curr + 1, right = 2 * curr + 2;
      if (left < i && arr[left] > arr[largest]) {
        largest = left;
      }
      if (right < i && arr[right] > arr[largest]) {
        largest = right;
      }
      if (largest == curr) {
        break;
      }
      std::swap(arr[largest], arr[curr]);
      curr = largest;
    }
  }
}

int main() {
  std::cout << "Correctness Tests\n";
  std::vector<int> test = {64, 34, 25, 12, 22, 11, 90};
  std::cout << "Original Array:  ";
  printFirst10(test);

  auto testWith = test;
  heapSortWithHeapify(testWith);
  std::cout << "With Heapify:    ";
  printFirst10(testWith);

  auto testWithout = test;
  heapSortWithoutHeapify(testWithout);
  std::cout << "Without Heapify: ";
  printFirst10(testWithout);
  std::cout << "\n";

  std::cout << "Performance Benchmarks";
  std::vector<size_t> testSizes = {10000, 50000, 100000};
  std::vector<std::pair<std::string, InputType>> testTypes = {
      {"Random Data", InputType::RANDOM},
      {"Sorted Data", InputType::SORTED},
      {"Reverse Sorted Data", InputType::REVERSE_SORTED}};
```

```
  for (const auto &[name, type] : testTypes) {
    std::cout << "\nTesting Pattern: " << name << "\n";
    for (size_t N : testSizes) {
      std::vector<int> original = generateTestCase(N, type);
      if (N == 10000)
        printFirst10(original);

      // Benchmark Heap Sort With Heapify
      auto dataWith = original;
      auto startWith = std::chrono::steady_clock::now();
      heapSortWithHeapify(dataWith);
      auto endWith = std::chrono::steady_clock::now();
      std::chrono::duration<double, std::milli> timeWith = endWith - startWith;

      // Benchmark Heap Sort Without Heapify
      auto dataWithout = original;
      auto startWithout = std::chrono::steady_clock::now();
      heapSortWithoutHeapify(dataWithout);
      auto endWithout = std::chrono::steady_clock::now();
      std::chrono::duration<double, std::milli> timeWithout =
          endWithout - startWithout;

      std::cout
          << "N=" << std::setw(6) << N << "|Heapify:" << std::setw(7)
          << std::fixed << std::setprecision(4) << timeWith.count() << " ms"
          << " (Valid:"
          << (std::is_sorted(dataWith.begin(), dataWith.end()) ? "Yes" : "No")
          << ")"
          << " |No Heapify:" << std::setw(7) << timeWithout.count() << " ms"
          << " (Valid:"
          << (std::is_sorted(dataWithout.begin(), dataWithout.end()) ? "Yes"
                                                                     : "No")
          << ")\n";
    }
  }

  return 0;
}
```

4.1 `util/test.hpp`

```
#pragma once

#include <cstddef>
#include <vector>

enum class InputType { RANDOM, SORTED, REVERSE_SORTED, DUPLICATES };

std::vector<int> generateTestCase(size_t size, InputType type);
void printFirst10(const std::vector<int> &arr);
```

4.2 `util/test.cpp`

```
#include "test.hpp"
#include <algorithm>
#include <iostream>
#include <numeric>
#include <random>

std::vector<int> generateTestCase(size_t size, InputType type) {
```

```cpp
  std::vector<int> arr(size);
  switch (type) {
  case InputType::RANDOM: {
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<int> dist(1, 1000000);
    for (size_t i = 0; i < size; ++i)
      arr[i] = dist(gen);
    break;
  }
  case InputType::SORTED: {
    std::iota(arr.begin(), arr.end(), 1);
    break;
  }
  case InputType::REVERSE_SORTED: {
    std::iota(arr.rbegin(), arr.rend(), 1);
    break;
  }
  case InputType::DUPLICATES: {
    std::fill(arr.begin(), arr.end(), 42);
    break;
  }
  }
  return arr;
}

void printFirst10(const std::vector<int> &arr) {
  size_t count = std::min(size_t(10), arr.size());
  for (size_t i = 0; i < count; i++) {
    std::cout << arr[i] << (i == count - 1 ? "" : ", ");
  }
  std::cout << "\n";
}
```

5 Terminal Output

```
$ clang++ -O3 main.cpp test.cpp -o heap && ./heap

Correctness Tests
Original Array:  64, 34, 25, 12, 22, 11, 90
With Heapify:    11, 12, 22, 25, 34, 64, 90
Without Heapify: 11, 12, 22, 25, 34, 64, 90

Performance Benchmarks
Testing Pattern: Random Data
727630, 432182, 692387, 770573, 169162, 143379, 664849, 423616, 584054, 136158
N= 10000|Heapify: 5.0486 ms (Valid:Yes) |No Heapify: 7.6016 ms (Valid:Yes)
N= 50000|Heapify:14.7019 ms (Valid:Yes) |No Heapify:11.5232 ms (Valid:Yes)
N=100000|Heapify:15.7194 ms (Valid:Yes) |No Heapify:20.6697 ms (Valid:Yes)

Testing Pattern: Sorted Data
1, 2, 3, 4, 5, 6, 7, 8, 9, 10
N= 10000|Heapify: 0.8066 ms (Valid:Yes) |No Heapify: 1.1012 ms (Valid:Yes)
N= 50000|Heapify: 4.3930 ms (Valid:Yes) |No Heapify: 6.6500 ms (Valid:Yes)
N=100000|Heapify: 8.9167 ms (Valid:Yes) |No Heapify:12.2823 ms (Valid:Yes)

Testing Pattern: Reverse Sorted Data
10000, 9999, 9998, 9997, 9996, 9995, 9994, 9993, 9992, 9991
N= 10000|Heapify: 0.8555 ms (Valid:Yes) |No Heapify: 1.1623 ms (Valid:Yes)
N= 50000|Heapify: 4.7529 ms (Valid:Yes) |No Heapify: 6.2991 ms (Valid:Yes)
N=100000|Heapify:10.2978 ms (Valid:Yes) |No Heapify:12.9879 ms (Valid:Yes)
```